

86. Sets aur Ranges [...]

Square brackets [...] ke andar kai characters ya character classes ka matlab: "diye gaye me se koi ek character".

Sets

[eao] = 'a', 'e' ya 'o' me se koi ek. Ise **set** kehte hain. Normal characters ke saath use kar sakte hain:

```
js
"Mop top".match(/[tm]op/gi);    // "Mop", "top"
```

Dhyan: set me kai characters hone ke bawajood match me wo **sirf ek** character ke barabar hota hai. Ye isliye koi match nahi deta:

```
js
"Voila".match(/V[oi]la/);    // null
```

Pattern kehta hai: [V], phir [oi] me se **ek**, phir [la]. To [Vo] ya [Vi] match hote.

Ranges

Square brackets me **character ranges** bhi ho sakti hain. [a-z] = [a] se [z], [0-5] = [0] se [5].

```
js
"Exception 0xAF".match(/x[0-9A-F][0-9A-F]/g);    // xAF
```

[0-9A-F] me 2 ranges hain: digit [0-9] ya letter [A-F]. Lowercase bhi chahiye to [a-f] jodo ([0-9A-Za-f]) ya [i] flag lagao.

Sets ke andar **character classes** bhi use ho sakti hain:

- [\w-]: word character ya hyphen
- [\s\d]: space character ya digit

Character classes kuch sets ke shorthands hain:

- \d = [0-9]
- \w = [a-zA-Z0-9_]
- \s = [\t\n\v\f\r] + kuch rare Unicode space characters

Example: multi-language (\w)

(\w) ([a-zA-Z0-9_]) hai, isliye Chinese hieroglyphs, Cyrillic waghera nahi dhundh sakta. Har language ke "word" characters ke liye Unicode properties se set banao:

```
js
```

```
[\p{Alpha}\p{M}\p{Nd}\p{Pc}\p{Join_C}]
```

- (Alphabetic) ((Alpha)): letters
- (Mark) ((M)): accents
- (Decimal_Number) ((Nd)): digits
- (Connector_Punctuation) ((Pc)): underscore (_) jaise
- (Join_Control) ((Join_C)): 2 special codes (200c), (200d) (ligatures me, jaise Arabic)

```
js
```

```
let regexp = /[\p{Alpha}\p{M}\p{Nd}\p{Pc}\p{Join_C}]/gu;  
"Hi 你好 12".match(regexp);    // H,i,你,好,1,2
```

IE me Unicode properties nahi hoti (XRegExp library ya sirf ek language ki range jaise ([a-я])).

Excluding ranges ([^...])

Shuruaat me caret (^) ho to matlab: diye gaye characters **ke alawa** koi bhi.

- ([^aeyo]): ('a', 'e', 'y', 'o') ke alawa koi bhi
- ([^0-9]): digit ke alawa, (\D) jaisa
- ([^\s]): non-space, (\S) jaisa

```
js
```

```
"alice15@gmail.com".match(/^[^\d\sA-Z]/gi);    // @ aur .
```

(...) ke andar escaping

Aam taur par special character literally chahiye to (\.) jaise escape karte hain. Par square brackets ke andar zyaadatar special characters **bina escape** ke use kar sakte hain:

- (. + ()) ko kabhi escape nahi karna
- Hyphen (-) shuru ya end me escape nahi (jahan range nahi banata)
- Caret (^) sirf shuru me escape (jahan exclusion ka matlab hota)
- Closing bracket (]) hamesha escape (agar wahi dhundhna ho)

Yaani jo cheez square brackets ke liye khaas matlab nahi rakhti wo bina escape ke chalti hai.

[.,] = dot ya comma.

js

```
let regexp = /[-().^+]/g;  
"1 + 2 - 3".match(regexp);    // +, -
```

"Just in case" escape karoge to bhi koi nuksaan nahi: `/[\-\\(\\)\.\\^\\+]/g`.

Ranges aur `u` flag

Set me surrogate pairs ho to sahi kaam ke liye `u` flag chahiye:

js

```
'x'.match(/[xy]/);    // ajeeb character [?] (galat: aadha character mila)  
'x'.match(/[xy]/u);    // x
```

Bina `u` ke engine `[xy]` ko 2 nahi, 4 characters maanta hai (har surrogate pair ke 2 aadhe hisse).

Range ke saath bhi: `[x-y]` bina `u` ke **error** deta hai (Invalid regular expression), kyunki range ka start code end se bada ban jaata hai.

js

```
'y'.match(/[x-z]/u);    // y
```

Tasks

- `Java[^script]`: `Java` me match nahi (koi character hi nahi bacha), `JavaScript` me **match** (`JavaS`), kyunki `S` uppercase hai aur `[^script]` me nahi hai, regexp case-sensitive).
- Time** `hh:mm` ya `hh-mm`: `\d\d[-:]\d\d` (hyphen shuru me hai, isliye escape nahi chahiye).